

(Task 1 of 15) In this tutorial, we will learn *more* about definitions.

0

(Task 2 of 15) What is the result of running this program?

Lispy | 

Lispy [Run ▶]

Python

```
(deffun (f x)
  (defvar y 1)
  (+ x y))
(+ (f 2) y)
```

```
def f(x):
    y = 1
    return x + y
print(f(2) + y)
```

error

2

You predicted the output correctly 🎉🎉🎉

3

`(+ (f 2) y)` is evaluated in the top-level block, where `y` is not defined. So, this program errors.

Click [here](#) to run this program in the Stacker.

(Task 3 of 15) What is the result of running this program?

Lispy | 

Lispy [Run ▶]

Scala 3

```
(defvar x 1)
(deffun (f y)
  (deffun (g)
    (defvar z 2)
    (+ x y z)))
(g)
(+ (f 3) 4)
```

```
val x = 1
def f(y : Int) =
  def g =
    val z = 2
    x + y + z
  g
println(f(3) + 4)
```

10

5

You predicted the output correctly 🎉🎉🎉

6

This program binds `x` to 1 and `f` to a function and then evaluates `(+ (f 3) 4)`. The value of `(+ (f 3) 4)` is the value of `(+ (g) 4)`, which is the value of `(+ (+ x y z) 4)`, which is the value of `(+ (+ 1 3 2) 4)`, which is 10.

Click [here](#) to run this program in the Stacker.

(Task 4 of 15) What is the result of running this program?

Lispy | 

(Task 4 of 15) What is the result of running this program?

Lispy [Run]

Scala 3

```
(defvar x 1)      val x = 1
(deffun (f)        def f =
  x)              x
(deffun (g)        def g =
  (defvar x 2)      val x = 2
  (f))             f
                  println(g)

(g)
```

1

8

You predicted the output correctly 🎉🎉🎉

9

(g) evaluates to (f), which evaluates to 1 because f and x are both defined in the same block, and f does *not* get the x defined inside g.

Click [here](#) to run this program in the Stacker.

(Task 5 of 15) What is the result of running this program?

Lispy [Run]

Lispy [Run]

Pseudo

```
(defvar x 1)      let x = 1
(deffun (f)        fun f():
  (defvar y 2)      let y = 2
  (deffun (g)        fun g():
    (+ x y))          return x + y
  (g))              end
(f))                return g()
                    end
                    print(f())
```

3

11

You predicted the output correctly 🎉🎉🎉

12

(f) returns the value of (g). The value of (g) is the value of (+ x y), which is (+ 1 2), which is 3.

Click [here](#) to run this program in the Stacker.

(Task 6 of 15) What is the result of running this program?

Lispy [Run]

Lispy [Run]

Python

```

(defvar x 1)      x = 1
(defun (f y)      def f(y):
  (defvar x 2)    x = 2
  (+ x y))        return x + y
                  print(f(0) + x)
(+ (f 0) x)

```

3

14

You predicted the output correctly 🎉🎉🎉

15

This program binds `x` to `1` and `f` to a function in the top-level block. It binds `x` to `2` in the body of `f`. So, `(f 0)` evaluates to `(+ 2 0)`, which is `2`, and `(+ (f 0) x)` evaluates to `(+ 2 1)`, which is `3`.

Click [here](#) to run this program in the Stacker.

(Task 7 of 15) When we see a variable reference (e.g., the `x` in `(+ x 1)`), how do we find its value, if any?

16

Look up in local environment

17

(Task 8 of 15) Variable references follow the hierarchical structure of blocks.

18

If the variable is defined in the current block, we use that declaration.

Otherwise, we look up the block in which the current block appears, and so on recursively. (Specifically, if the current block is a function body, the next block will be the block in which the function definition is; if the current block is the top-level block, the next block will be the **primordial block**.)

If the current block is already the primordial block and we still haven't found a corresponding declaration, the variable reference errors.

The primordial block is a non-visible block enclosing the top-level block. This block defines values and functions that are provided by the language itself.

Any feedback regarding these statements? Feel free to skip this question.

19

(You skipped the question.)

20

(Task 9 of 15) Now please scroll back and select 1-3 programs that make the point that	21
If the variable is declared in the current block, we use that declaration. You don't need to select <i>all</i> such programs.	
(You selected 2 programs)	22
Okay. How do these programs (7,13) support the point?	23
Show use of inner variable not the outer one	24

(Task 10 of 15) Please select 1-3 programs that make the point that	25
Otherwise, we look up the block in which the current block appears, and so on recursively. You don't need to select <i>all</i> such programs.	
(You selected 2 programs)	26
Okay. How do these programs (7,13) support the point?	27
Same as above	28

(Task 11 of 15) Please select 1-3 programs that make the point that	29
If the current block is already the primordial block and we still haven't found a corresponding declaration, the variable reference errors. You don't need to select <i>all</i> such programs.	
(You selected 1 programs)	30
Okay. How does this program (1) support the point?	31
Shows directly	32

(Task 12 of 15) The scope of a variable is the region of a program where we can refer to	33
--	----

(Task 12 of 15) The **scope** of a variable is the region of a program where we can refer to the variable. Technically, it includes the block in which the variable is defined (including sub-blocks) *except* the sub-blocks where the same name is re-defined. When the exception happens, that is, when the same name is defined in a sub-block, we say that the variable in the sub-block **shadows** the variable in the outer block.

We say a variable reference (e.g., "the `x` in `(+ x 1)`") is **in the scope of** a declaration (e.g., "the `x` in `(defvar x 23)`") if and only if the former refers to the latter.

Any feedback regarding these statements? Feel free to skip this question.

34

(You skipped the question.)

35

(Task 13 of 15) Please select all statements that apply to the following program:

36

```
(defvar x 45)
(defun (f)
  (defun (g)
    (+ x 1))
  (defvar x 67)
  (+ x 2))
(f)
(+ x 3)
```

- ☒ The scope of the top-level `x` includes the whole program.
- ☒ The scope of the top-level `x` includes `(+ x 3)`.
- ☐ The top-level `x` shadows the `x` defined in `f`.
- ☐ The `x` in `(+ x 1)` is in the scope of the `x` defined in `f`.
- ☒ The `x` in `(+ x 2)` is in the scope of the `x` defined in `f`.
- ☐ The `x` in `(+ x 3)` is in the scope of the `x` defined in `f`.

37

You should also have chosen The `x` in `(+ x 1)` is in the scope of the `x` defined in `f`.

38

You should NOT have chosen The scope of the top-level `x` includes the whole program.

The scope of the top-level `x` includes the whole program *except* the body of `f`. The scope of the `x` defined in `f` is the body of `f`.

The `x` defined in `f` shadows the top-level `x` rather than the other way around.

(Task 14 of 15) Here is a program that confused many students

 Lispy | 

Lispy [Run]

Pseudo

```
(defvar x 1)    let x = 1
```

```

(defun (foo)
  x)
(defun (bar)
  (defvar x 2)
  (foo))

(bar)

```

```

fun foo():
  return x
end
fun bar():
  let x = 2
  return foo()
end
print(bar())

```

Please

1. Run this program in the stacker by clicking one of the [\[Run\]](#) buttons above;
2. The stacker would show how this program produces its result(s);
3. Keep clicking [» Next](#) until you reach a configuration that you find particularly helpful;
4. Click [Share This Configuration](#) to get a link to your configuration;
5. Submit your link below;

https://smol-tutor.xyz/stacker/?syntax=Lispy&randomSeed=smol-
 tutor&hole=%E2%80%A2&nNext=2&program=%0A%28defvar+x+1%29%0A%28deffun+
 %28foo%29%0A++x%29%0A%28deffun+%28bar%29%0A++
 %28defvar+x+2%29%0A++
 %28foo%29%29%0A%0A%28bar%29%0A&readOnlyMode=

40

(Task 15 of 15) Please write a couple of sentences to explain how your configuration explains the result(s) of the program.

41

Shows that when foo is called it evaluates in an environment inherited from top-level and not from the bar environment

42

Let's review what we have learned in this tutorial.

43

Variable references follow the hierarchical structure of blocks.

If the variable is defined in the current block, we use that declaration.

Otherwise, we look up the block in which the current block appears, and so on recursively. (Specifically, if the current block is a function body, the next block will be the block in which the function definition is; if the current block is the top-level block, the next block will be the **primordial block**.)

If the current block is already the primordial block and we still haven't found a

corresponding declaration, the variable reference errors.

The primordial block is a non-visible block enclosing the top-level block. This block defines values and functions that are provided by the language itself.

The **scope** of a variable is the region of a program where we can refer to the variable. Technically, it includes the block in which the variable is defined (including sub-blocks) *except* the sub-blocks where the same name is re-defined. When the exception happens, that is, when the same name is defined in a sub-block, we say that the variable in the sub-block **shadows** the variable in the outer block.

We say a variable reference (e.g., "the `x` in `(+ x 1)`") is **in the scope of** a declaration (e.g., "the `x` in `(defvar x 23)`") if and only if the former refers to the latter.

You have finished this tutorial 🎉🎉🎉

Please [print](#) the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1780299495669